SLiM Workshop Series #22: SLiM at the Command Line

Copyright © 2020–2025 Benjamin C. Haller. All rights reserved.

1. Start a new model in SLiMgui

Run SLiMgui, and close any existing model windows. Press command-N to make a new model window. Adapt the model to the following code, or just delete it and enter this model:

```
initialize() {
   initializeMutationRate(1e-7);
   initializeMutationType("m1", 0.5, "f", 0.0);
   initializeGenomicElementType("g1", m1, 1.0);
   initializeGenomicElement(g1, 0, 99999);
   initializeRecombinationRate(1e-8);
}
1 early() {
   sim.addSubpop("p1", 500);
}
5000 late() { sim.outputFixedMutations(); }
```

Save the model by pressing command-S or choosing **Save** from the **File** menu. Save it to the Desktop, with a filename of `rep.slim`. Our goal in this exercise is to see how we can adapt this model to be run at the command line, as a first step toward running it on a computing cluster.

2. Run the model at the command line

We want to run this model at the command line, so switch to **Terminal.app**. This might be in the Dock; if not, it should be in `/Applications/Utilities`. Its icon should look like this:

Terminal provides you with a command-line prompt where you can run Unix commands. Once it is launched, type `cd ~/Desktop` and press return; this changes the current directory in Terminal to the Desktop. Then type: `slim rep.slim` and press return. This runs the `slim` command-line tool, which can execute SLiM models on macOS, Linux, and Windows. The model should run in a second or so, producing output directly into Terminal. So far, so good!

3. Adapt the model to use defined parameters

We want to be able to control the parameters of the model with command-line arguments, so we need to change the model to use defined parameters. Modify the script to look like this:

```
initialize() {
   defineConstant("MU", 1e-7);
   defineConstant("R", 1e-8);
   defineConstant("N", 500);

   initializeMutationRate(MU);
   initializeMutationType("m1", 0.5, "f", 0.0);
   initializeGenomicElementType("g1", m1, 1.0);
   initializeGenomicElement(g1, 0, 99999);
   initializeRecombinationRate(R);
}
```

```
1 early() {
   sim.addSubpop("p1", N);
}
5000 late() { sim.outputFixedMutations(); }
```

This is the same except that constants `MU`, `R`, and `N` are now defined at the top, and are then used in the script for the mutation rate, recombination rate, and population size.

Switch back to Terminal. Press command-K to clear the old output from the window, to provide a clean slate for working. Now, you will want to run the command `slim rep.slim` again, but you shouldn't need to retype it; if your machine is correctly configured, you should be able to press the up-arrow key to get the previous command in the command history, and then simply press return to execute it again.

4. Pass a random number seed to `slim`

Re-run the model several times by pressing up-arrow and then return. Note that each run produces different output; this is because the random number seed used by `slim` is different each time (based upon the system clock at the start of the run). When doing replicated runs on a computing cluster, we typically want to use known seed values so that we can reproduce any particular run of the model (in case that run does something particularly interesting, or exhibits a bug – or just for scientific reproducibility of our results). We can easily specify a seed value for `slim` to use. In Terminal, type:

```
slim -s 1 rep.slim
```

and hit return. Once that run has completed, hit up-arrow and return a couple of times to run it repeatedly; it should produce the same output every time. Notice that at the top of each run's output, it says:

```
// Initial random seed:
1
```

That tells us the initial random number seed used for that run – a value of 1, as requested by the `-s` command-line option.

5. Adapt the model to accept command-line parameters

We've got the model using the parameters `MU`, `R`, and `N`, but their values are hard-coded in the script. We'd like to be able to change those values at the command line, so that we can run the same model with a variety of parameter values easily. However, we still want to be able to run the model in SLiMgui, too, using default values for the parameters. To achieve this, go back to SLiMgui and change the `initialize()` callback to start this way:

```
initialize() {
   if (exists("slimgui")) {
      defineConstant("MU", 1e-7);
      defineConstant("R", 1e-8);
      defineConstant("N", 500);
   }
   ...
```

The `...` represents the rest of the callback, of course; don't type "…". Now recycle and run in SLiMgui; the model should run as before, using the defined parameter values. How does this work? There is a global variable named `slimgui` that is defined only when a model is running inside SLiMgui; it provides the ability to control SLiMgui from the model script. The `exists()` function

checks for the existence of any named symbol; if a symbol named `"slimgui"` exists, we're running under SLiMgui and therefore ought to define the model's parameters.

Save the model in SLiMgui with the change you made above. Now go back to Terminal, and press up-arrow and return the run the model again. You will get an error like this:

```
ERROR (EidosSymbolTable::_GetValue): undefined identifier MU.

Error on script line 8, character 24:

   initializeMutationRate(MU);
                          ^^
```

This is `slim` telling us there's a problem. It tells us the line and character in the script where the problem occurred (line 8, character 24), and shows us the offending line with carets, `^^`, under the spot where the error was encountered. It also tells us what the error was ("`undefined identifier MU`"), and – mostly for use by developers of SLiM itself – the internal function where the error occurred (`EidosSymbolTable::_GetValue`).

So, great – this makes sense, because we've just changed the model to expect parameter values to be defined at the command line. In Terminal, type this command line and hit return:

```
slim -s 1 -d MU=1e-7 -d R=1e-8 -d N=500 rep.slim
```

The model should run nicely. We're still passing a seed of `1`, but now we are also passing definitions of the constants `MU`, `R`, and `N` using the `-d` command-line flag. Each use of `-d` is followed by a symbol name, an equals sign, and a value. Try running the model with different parameters, like:

```
slim -s 1 -d MU=1e-6 -d R=1e-8 -d N=500 rep.slim
```

This run will probably produce quite a few fixed mutations at the end, given the elevated mutation rate.

As an aside: an alternative to using `exists("slimgui")` to guard the definition of a constant is to use `!exists(CONSTANT)` instead. For example, we could write:

```
if (!exists("MU"))
   defineConstant("MU", 1e-7);
```

to define `MU` as `1e-7` if, and only if, `MU` does not already exist. This will define `MU` as `1e-7` when running under SLiMgui, but it will *also* define `MU` as `1e-7` if it is not otherwise defined at the command line. This approach has its pros and cons; it provides a default value both in SLiMgui and at the command line, but it *also* means that if you were to accidentally mistype `-d MH=1e-6` at the command line, the model would run with `MU` as `1e-7`, not `1e-6`, with no error reported! Sometimes it is better to require something to be explicit, for safety, rather than trying to be too smart with default values and default behaviors. You can mix both styles in the same model, of course, or you could perhaps read in parameter defaults from a JSON file on disk. You can do whatever you want, because you are the script-writers, and you are the dreamers of the dream.

6. Scale the runtime by the population size

We'd like the model to run for a number of ticks that is proportional to the population size, `N`. This is often desirable because the expected time for neutral mutations to fix is proportional to `N`; if you increase `N` without scaling, the model will just be getting started when it ends. This is easy to see with our model in Terminal; try:

```
slim -s 1 -d MU=1e-6 -d R=1e-8 -d N=5000 rep.slim
```

Before, when we used a mutation rate of `1e-6`, we got tons of fixed mutations at the end of the run. Now, with the same mutation rate but a population size ten times larger, you probably got none at all; there just wasn't enough time for neutral mutations to drift to fixation.

Go back to SLiMgui and add this line, immediately after the other defined constants:

```
defineConstant("G", N*10);
```

Now `G` is the tick we want the model to execute to. To use that constant, just change the final output event to look like this, in order to run in tick `G`:

```
G late() { sim.outputFixedMutations(); }
```

Save and run the model in SLiMgui, and it should still run through tick `5000`. If you change the definition of `N` to `50`, however – do that now, and recycle and run – it should then run only through tick `500`.

Complex models with many different events that all need to run at different parameter-driven times can use different constants for each scheduled time, but that can get a bit clunky. Instead, note that it is also legal to write this, for example:

```
N*10 late() { sim.outputFixedMutations(); }
```

So you don't necessarily need that constant `G`; if you always want an event to run in tick `N*10`, you can just put that expression into your script. You can use pretty much any Eidos expression to schedule a tick – not just `G` or `N*10`, but `N*M+1000`. You can call a built-in function or a user-defined function in a tick schedule expression – even ones that use the random number generator! So, for example, something like:

```
rdunif(1, min=N, max=N*2) late() { ... }
```

would be legal, scheduling the event at a random tick in the range [`N`, `2N`]. A vector of integers can be used to schedule an event in multiple ticks, and the ticks don't need to be contiguous, so `c(1,10,100,1000)` would be a legal tick schedule, as would `seq(from=20, to=1000, by=20)`.

Given all this, it would be perfectly reasonable to get rid of the constant `G` and use `N*10` as the scheduled tick for the model's end. But suppose we want to keep `G`, so that we can do some command-line runs where `G` is `N*10`, and other runs where `G` is `N*20`? How do we define `G` to be `N*10` or `N*20` at the command line?

7. Expressions in command-line definitions

Try running the previous section's model at the command line with the same command as before:

```
slim -s 1 -d MU=1e-6 -d R=1e-8 -d N=5000 rep.slim
```

You will get a rather verbose error about how the final script block never executed because `G` was never defined. We've defined `G` inside the `if (exists())` block, so it is only defined when running inside SLiMgui. If we *always* wanted `G` to be `N*10`, we could simply move the definition outside of that block so that `G` is thusly defined in all cases. However, suppose we sometimes want `G` to be `N*10`, and sometimes `N*20`, so that we can check whether `N*10` is long enough to produce equilibrium results. We can actually use arbitrary Eidos expressions in `-d` defines, so we can do this:

```
slim -s 1 -d MU=1e-6 -d R=1e-8 -d N=2500 -d G="N*10" rep.slim
```

That will take a little while to run – large-ish population size and high mutation rate – but in less than a minute you should get a big dump of fixed mutations.

Notice that we put the expression `N*10` in quotes. This is not defining `G` to be an Eidos string; the quotes actually get stripped off by the Terminal shell before Eidos ever even sees them, so as far as Eidos is concerned `G` has been defined to be `N*10`. The reason the quotes are there is that the `*` character has a meaning in Terminal, as a filename wildcard; we need to quote the definition to avoid confusing Terminal. If you want to be safe, you can simply always quote the definitions of all `-d` constants so that the Terminal shell never interferes with them. (If you ever do want to do a command-line definition of an Eidos string, you will need to quote your quotes, so that the shell doesn't mess things up! See section 21.2 of the SLiM manual for discussion of this wrinkle.)

You can define other constants with expressions too; many population-genetics models want to use `RHO` and `THETA` as parameters, and so such models might calculate SLiM's per-base-position mutation and recombination rates based upon `RHO`, `THETA`, and `N`. Definitions are processed in the order in which they are supplied on the command line, so later definitions can depend upon earlier definitions (as `G`'s definition depends upon `N` here).

8. Running a batch of simulations with a shell script

Now we'd like to run this model multiple times, with different parameter values. We're going to do this with a shell script that runs in the Unix shell Bash. Getting things set up will take a few steps. First of all, go to Terminal, execute `cd ~/Desktop` to go to the Desktop directory if you're not already there, and type the command:

```
touch run_reps.sh
```

This creates a new file named `run_reps.sh`, which will become our sublaunch shell script. Now, still in Terminal, on macOS type:

```
open -e run_reps.sh
```

This asks Terminal to open the file `run_reps.sh` in TextEdit, the default text editor in macOS (of course feel free to use a different plain-text editor instead, such as Atom or Sublime Text). Linux users might try `gedit` instead of `open -e`; it's a decent plain-text editor, but you may need to install it with a command like `sudo apt-get install gedit`. Your machine should switch to TextEdit (or `gedit`, or whatever), launching it if it isn't already running, and should then open a new empty window titled `run_reps.sh`. In that window, enter the following text:

```
echo "foo"
```

(Make sure the straight quotes haven't been replaced with curly quotes; if they have, find and turn off the "smart quotes" pref in your editor.) Save the file. This rudimentary shell script simply prints "foo" to the output, using the Bash command `echo`.

Let's go back to Terminal now and run the script, by executing:

```
/bin/bash run_reps.sh
```

If all has gone according to plan, you should get the expected output:

```
foo
```

Now we're ready to expand this script into a proper sublaunch script. Go back to the script file in your editor and replace the script with the following:

```
slim -s 1 -d MU=1e-7 -d R=1e-8 -d N=100 -d G="N*10" rep.slim
```

Now, executing `/bin/bash run_reps.sh` should result in that SLiM run being sublaunched; try that now to make sure it works.

We'd like to do multiple runs with different values for `N`. To achieve that, add a `for` loop:

```
for N in 100 200 500 1000 ; do
   echo "Running with N =" ${N}:
   slim -s 1 -d MU=1e-7 -d R=1e-8 -d N=${N} -d G="N*10" rep.slim
   echo
done
```

This is a bit different from Eidos, but it should be clear enough what's going on. For help with the syntax of Bash shell scripts, there's lots of information online including free tutorials and such, so we won't go into any detail here. Suffice to say that we're looping over multiple values for `N`, printing a line above each run's output, and then sublaunching the run with the current loop iteration value of `N` using the Bash syntax `${N}` to refer to the value of variable `N`.

Execute this, with `/bin/bash run_reps.sh` as usual, and you should see the four model runs occur in succession.

If you would prefer to sublaunch your simulations using R or Python, the slim-community organization on GitHub has examples of sublaunch scripts in those languages, in a repository at https://github.com/slim-community/slim-sublaunching; you should be able to adapt them without much trouble.

9. Redirecting output to separate files

It's not really very convenient having the output from all the runs dumping right into Terminal; we'd like it to be placed into separate files instead. To achieve this, we just need to add a redirection operator to the `slim` command, like so:

```
slim -s 1 -d MU=1e-7 -d R=1e-8 -d N=${N} -d G="N*10" rep.slim > out_${N}.txt
```

The `>` symbol tells Bash to redirect the output from `slim` to the following file, where the filename `out_${N}.txt` produces files named `out_100.txt`, `out_200.txt`, `out_500.txt`, and `out_1000.txt`. Try running this script now, again with `/bin/bash run_reps.sh`, and verify that the output files have been created as expected.

EXERCISE: One often wants to run multiple replicates for each parameter value, so that you don't get just a single result for a given parameter value (which might be idiosyncratic), but instead a distribution of results. First, create a new folder on your Desktop, named `RepWork`, drag the `rep.slim` script and `run_reps.sh` script inside it, and `cd ~/Desktop/RepWork` in Terminal so you don't clog up the Desktop with too many files. Then, modify the shell script to do three replicates for each value of `N`, by adding a nested `for` loop over values `1 2 3` with loop index variable `SEED`. Pass the value of `SEED` to `slim` as the random number seed with `-s`, and make the redirected output go to files with filenames like `out_Nxxxxx_SEEDxxxxx.txt`, where `xxxxx` should be value of the relevant variable. Run the script to confirm that it does all the replicate runs expected.

At this point you've got a sublaunch script that you could use on your local machine, perhaps running tasks simultaneously in the background by putting " `&`" at the end of the sublaunch line (another Bash feature – try it!). To run on a computing cluster, you would typically not run `slim` directly, but would instead use a job-submission command like `qsub` to schedule jobs on cores. The details of doing that will depend on your cluster and the scheduler it uses, so it's beyond the scope of this workshop, but given the tools we've seen in this worksheet, it shouldn't be too hard to get it working.

9. BEEP! With extra time, do some Googling about Bash shell script syntax to find out more, or read SLiM manual section 21.3 on other command-line flags for `slim`, beyond what we've seen here.